# Threads of Life

Kaavish Report
presented to the academic faculty
by


| | |
|---|---|
| Muhammad Shayan Wasif | mw08071 |
| Muhammad Hayyan Khan | mk08321 |
| Naveen Zehra Zaidi | nz08093 |
| Incia Saleem | is08311 |


In partial fulfillment of the requirements for
*Bachelor of Science*
Computer Science

**Dhanani School of Science and Engineering**

Habib University
Spring 2022

Copyright © 2019 Habib University

# Threads of Life

This Kaavish project was supervised by:

My Internal Supervisor
Faculty of Computer Science
Habib University

Approved by the Faculty of Computer Science on _____.

# Acknowledgements

We would like to sincerely thank the CS faculty, for their continous guidance, motivation, and encouragement throughout this project. We would especially like to thank our faculty supervisor, Professor Usman Arif, without whom, we would not have been able to achieve the progress we have in our Kaavish. His insights and feedbacks played a pivotal role in shaping our year-long journey of this final project. We are also grateful to Amanah Ventures for providing us with the opportunity, resources, and global industry exposure that played a significant role in turning our ideas into a a final product.

# Abstract

**Threads of Life** is a 2D narrative-driven interactive adventure designed to address the limitations of traditional moral choice systems in video games, which often rely on rigid binary mechanics and predefined dialogue trees. This project introduces an AI-driven gameplay loop based on the concept of **consequential emergence**, allowing players to respond to morally ambiguous scenarios using freeform natural language through either text or speech-to-text (STT) interfaces.

The system utilizes a modular client-server architecture, where a **Unity 2D** frontend manages player interaction and environment rendering, while a local **Python Flask** backend performs real-time moral evaluation using the **Gemini 2.5 Flash API**. Player responses are analyzed across four moral dimensions—prosocial behavior, activeness, selflessness, and engagement—and categorized into behavioral outcomes that dynamically unlock physical paths within the game world.

Experimental evaluation of various NLP models, including BERT and BART-Large-MNLI, demonstrated that **Gemini 2.5 Flash** provided superior performance with 100% accuracy on the test dataset and an acceptable average latency of 2793 ms. To ensure gameplay continuity, a keyword-based fallback system was implemented for instances of API latency or failure. By integrating conversational AI with spatial narrative design, **Threads of Life** demonstrates a shift toward more expressive, personal, and reflective ethical gameplay experiences.

# Contents


**1 Introduction** **v**
1.1 Problem Statement . . . v
1.2 Proposed Solution . . . v
1.3 Intended Users . . . vi
1.3.1 Primary Users . . . vi
1.3.2 Secondary Users . . . vi
1.4 Project Gantt Chart and Deliverables . . . vi
1.5 Key Challenges . . . vii

**2 Literature Review** **viii**
2.1 Literature Review . . . viii
2.1.1 Ethical Pedagogy in Games . . . viii
2.1.2 Speech Interaction and Accessibility in Games . . . viii
2.1.3 Natural Language Processing for Moral Sentiment . . . ix
2.1.4 Interactive Narrative Design . . . ix
2.1.5 Research Gaps . . . x
2.1.6 Research Contribution . . . x

**3 Software Requirement Specification (SRS)** **xii**
3.1 Functional Requirements . . . xii
3.2 Non-Functional Requirements . . . xiv
3.3 External Interfaces . . . xvi
3.3.1 User Interfaces . . . xvi
3.3.2 Application Program Interface (API) . . . xvi
3.3.3 Hardware/Communication Interfaces . . . xvii
3.4 Use Cases . . . xvii
3.5 Datasets . . . xviii
3.6 System Diagram . . . xix

**4 Software Design Specification (SDS)** **xx**
4.1 Software Design . . . xx
4.1.1 System Architecture Overview . . . xx
4.1.2 Main Components of the System . . . xx

4.1.3 Relationships Among Components . . . xxiii
4.2 Data Design . . . xxiii
4.2.1 Runtime and Persistent Data Structures . . . xxiv
4.2.2 Logical Data Entities . . . xxiv
4.2.3 Save File Structure . . . xxiv
4.3 Technical Details . . . xxv
4.3.1 Text Input Processing Pipeline . . . xxv
4.3.2 Voice Input Processing Pipeline . . . xxvi
4.3.3 Save and Load Process . . . xxvi
4.3.4 Input Validation Algorithm . . . xxvi
4.3.5 Moral Classification Algorithm . . . xxvii
4.3.6 Keyword Fallback Algorithm . . . xxvii
4.3.7 Branch Unlocking Algorithm . . . xxviii
4.3.8 Design Principles Used . . . xxviii
4.3.9 Component Interfaces . . . xxviii

**5 Methodology, Experiments and Results OR Testing and Analysis xxx**
5.1 Overview of AI Model Evaluation . . . xxx
5.2 Models Evaluated . . . xxx
5.3 Experimental Dataset . . . xxxi
5.4 Evaluation Methodology . . . xxxii
5.5 Experimental Results . . . xxxii
5.6 Analysis of Results . . . xxxii
5.7 Final Model Selection . . . xxxiii
5.8 Limitations . . . xxxiii
5.9 Conclusion . . . xxxiii

**6 Conclusion and Future Work xxxiv**

**7 Reflection xxxv**

**References xxxvi**

# List of Figures

3.1 High-Level System Architecture of Threads of Life . . . . . xix

4.1 Overall system architecture of Threads of Life . . . . . xx

# List of Tables

4.1 Summary of interfaces between major system components . . . . . xxix

5.1 Model comparison results . . . . . xxxii

# 1. Introduction

## 1.1 Problem Statement

In the modern video game industry, ethical decision-making is often simplified into rigid and binary systems, such as basic "good" versus "evil" meters or predefined A/B/C dialogue trees. These mechanics restrict player expression and frequently encourage players to optimize outcomes for rewards rather than engage in genuine moral reflection. As a result, many games prioritize fast-paced action and superficial engagement over meaningful opportunities for empathy, introspection, and personal growth.

Therefore, there is a need for interactive systems that present players with emotionally complex and morally ambiguous situations, allowing them to respond freely rather than selecting from a limited set of predefined options. Such systems could encourage deeper reflection on the consequences of one's actions and create a more immersive and meaningful narrative experience.

## 1.2 Proposed Solution

*Threads of Life* is a 2D narrative-driven interactive adventure designed to address the limitations of traditional moral choice systems. The game introduces an AI-driven gameplay loop based on the concept of *consequential emergence*. Players experience a cyclical journey of life, encountering moral dilemmas at various scenario "gates" within the game world.

Instead of choosing from fixed dialogue options, players respond to situations using natural language through either text input or a Speech-to-Text (STT) module. A local Python backend connected to the Gemini 2.5 Flash API evaluates the player's response in real time across dimensions such as empathy, prosocial behavior, and level of engagement. Based on this evaluation, the response is categorized into general behavioral outcomes (for example: A – Heroic, B – Neutral/Moderate, or C – Avoidant).

These classifications directly influence the game environment by unlocking different physical paths in the 2D world, allowing the player's moral decisions to shape their progression. The experience is further enhanced through immersive audio gen-

erated using LMNT text-to-speech for non-playable characters (NPCs), as well as dynamic user interface feedback that reflects the consequences of the player's choices.

## 1.3 Intended Users

### 1.3.1 Primary Users

The primary audience consists of youth and young adult players (approximately ages 12–20) who enjoy narrative-driven games such as role-playing games (RPGs) and story-based adventures. The game aims to provide a safe and reflective space in which players can explore personal values, navigate moral dilemmas, and observe the broader consequences of their decisions.

### 1.3.2 Secondary Users

Secondary users include educators, psychological researchers, and game design students interested in moral cognition, ethics education frameworks (such as EPIC), and the integration of Large Language Models (LLMs) and Speech-to-Text interfaces in Human–Computer Interaction (HCI) systems.

## 1.4 Project Gantt Chart and Deliverables

hayyan gant chart dalo

The project follows a structured development timeline with an estimated total effort of approximately 500 person-hours. The goal is to produce a polished Minimum Viable Product (MVP) featuring around total 30 minutes of gameplay if you restart and explore all possibilities.

**Deliverable 1: Story and System Architecture (Completed)** Finalization of narrative scenarios, user interface architecture, and the integration of a local Python Flask server connected to the Gemini API. Basic 2D movement and environmental systems were also implemented.

**Deliverable 2: Prototype Level (Completed)** Development of a playable first level including character movement, scenario gates, and real-time branching paths determined by AI evaluation.

**Deliverable 3: Alpha and Beta Builds (Completed)** Integration of the remaining narrative scenes, implementation of immersive user interface elements (such

as visual feedback and timed response bars), and refinement of LMNT-based dynamically generated text-to-speech for NPC interactions.

**Deliverable 4: Final Polish (In progress)** Integration of the independent Speech-to-Text module for voice-based player input, finalization of character voice output, testing, performance optimization, and preparation of the final project report demonstrating major decision points and four distinct endings.

## 1.5 Key Challenges

**AI Latency and Reliability** Maintaining seamless interaction between the Unity 2D client and the local Python backend connected to an external LLM (Gemini 2.5 Flash) requires efficient response times. Ensuring that natural language processing occurs without disrupting gameplay immersion remains a key technical challenge. Reliability was a core challenge during development. We rigorously evaluated various AI models using performance metrics such as accuracy, latency, and F1 score to ensure our final selection was data-driven.

**Speech-to-Text Processing** Capturing audio within Unity and converting it into accurate text input introduces difficulties related to ambient noise, speech clarity, and transcription errors. Ensuring that the AI system receives a reliable representation of the player's intent is critical for consistent evaluation.

**Translating Sentiment into Physical Game Branches** Unlike traditional text-based narrative systems, this project relies on dynamically altering a physical 2D game environment. This requires careful management of colliders, triggers, and branching paths within Unity so that the environment updates smoothly based on AI-generated output categories.

**Scope Management** Narrative systems with branching consequences can easily grow exponentially in complexity. To maintain feasibility, the MVP is limited to three key stages, 10 possible decision scenarios and four distinct endings while still aiming to provide a meaningful and engaging player experience.

# 2. Literature Review

## 2.1 Literature Review

Recent research has explored the potential of video games as platforms for ethical learning and reflective decision-making. At the same time, advancements in conversational interfaces, speech-to-text (STT) technologies, and natural language processing (NLP) have expanded the possibilities for more expressive and immersive player interactions. This section reviews prior work related to ethical pedagogy in games, conversational AI and accessibility, moral sentiment analysis using NLP, and narrative design in interactive systems.

### 2.1.1 Ethical Pedagogy in Games

Researchers have increasingly emphasized the value of video games as tools for developing ethical reasoning and reflective thinking. Instead of presenting rigid moral codes, ethical game design aims to create environments in which players can explore moral dilemmas and observe the consequences of their decisions.

Schrier highlights that games can provide safe spaces for players to experiment with ethical decision-making through role-play and simulated consequences [7]. Building on this idea, the EPIC framework proposed by Schrier and Bachen outlines structured strategies for using video games in ethics education, emphasizing role-playing, ethical reflection, and consequence-based learning [7].

Earlier studies also highlighted limitations in existing educational simulations. Brennan and van der Waal examined business and management simulation games and found that many fail to meaningfully incorporate ethical ambiguity, often relying instead on point-based reward systems that reduce ethical decisions to optimization problems [2]. These findings suggest the need for systems that encourage deeper moral reflection rather than simplistic reward-driven gameplay.

### 2.1.2 Speech Interaction and Accessibility in Games

Recent developments in conversational interfaces have introduced new possibilities for player interaction in digital games. Speech-based interaction systems allow players to communicate with games using natural language, which can both improve

accessibility and enhance immersion.

Aguirre-Peralta et al. demonstrate the feasibility of speech-to-text control systems for video games using convolutional neural network (CNN) models for speech recognition [1]. Their research shows that voice-driven interaction can provide alternative control mechanisms for players with motor disabilities, making games more accessible.

Similarly, Frish et al. explore the integration of speech interaction with non-playable characters (NPCs) in digital games [4]. Their findings suggest that conversational interaction can significantly increase player immersion and create more natural engagement with virtual environments. However, the authors also highlight several challenges associated with speech interaction, including speech recognition errors, privacy concerns, and the cognitive load required for players to communicate verbally with digital systems.

### 2.1.3 Natural Language Processing for Moral Sentiment

Advances in natural language processing have enabled computational systems to detect moral reasoning and ethical sentiment within human language. Transformer-based language models have demonstrated the ability to analyze complex moral expressions in textual data.

Preniqi et al. introduce MoralBERT, a fine-tuned language model designed to capture moral values expressed in social discussions [5]. The model identifies ethical dimensions within language, enabling more nuanced analysis beyond traditional positive or negative sentiment classification. Similar research on models such as $ME^2$-BERT explores the integration of contextual and emotional information to improve moral prediction capabilities.

While these models are often applied to analyze moral discourse in online discussions, the same principles can be adapted for interactive systems such as video games. By analyzing player responses through language models, game systems can dynamically interpret player intentions and incorporate them into gameplay consequences.

### 2.1.4 Interactive Narrative Design

Interactive narrative design is a central component of story-driven games. Traditional narrative systems frequently rely on branching dialogue structures where players choose between predefined options that lead to predetermined outcomes. While such systems provide limited player agency, they may struggle to maintain narrative

coherence when players are given more expressive forms of interaction.

Research in computational narrative models suggests that interactive systems should dynamically respond to player actions in order to maintain coherent storytelling structures. Game development platforms such as the Unity engine provide flexible architectures for implementing dynamic gameplay systems that integrate narrative progression with environmental mechanics [3].

From a theoretical perspective, Rockwell introduces the concept of the chronotope in the analysis of computer games [6]. The chronotope describes the relationship between time and space in narrative media and highlights how spatial design can shape the player's interpretation of events within a game. In interactive environments, spatial elements such as branching paths and environmental transitions can therefore represent narrative consequences in meaningful ways.

### 2.1.5 Research Gaps

Despite significant progress in ethical game design and interactive AI systems, several limitations remain in current research. First, many speech-to-text systems developed for games primarily focus on command-based interaction rather than expressive dialogue. As a result, there is limited research exploring how spoken player responses can influence complex narrative outcomes.

Second, current NLP research on moral sentiment analysis is typically applied to static datasets rather than real-time player interactions within game environments. This limits the practical integration of these models into interactive narrative systems.

Third, traditional interactive narrative structures often rely on predefined branching dialogue trees that restrict player agency and expression. Such systems do not fully support the dynamic interpretation of natural language responses.

### 2.1.6 Research Contribution

This project aims to address these limitations by exploring the integration of speech-based interaction and natural language analysis within an ethical decision-making game environment. The proposed system allows players to respond to narrative scenarios using natural language through text or speech input.

Player responses are analyzed using large language models to identify ethical intent and sentiment. Instead of presenting predefined dialogue options, the system dynamically interprets player input and translates it into gameplay consequences within the game environment. By combining conversational interaction, moral sen-

timent analysis, and spatial narrative design, the project contributes toward the development of more expressive and immersive ethical gameplay systems.

# 3. Software Requirement Specification (SRS)

This chapter provides detailed specifications of the system under development.

## 3.1 Functional Requirements

This section describes each function/feature provided by the system. The functions are logically grouped into five modules based on their purpose and mode of operation.

- Module 1: Narrative Engine
  - Story Progression Control
    - Load the current scene and transition to the next based on prior player choices.
    - Manage reincarnation cycles, carrying moral data across levels to simulate karmic continuity.
  - Branching Path Management
    - Unlock physical pathways (A, B, or C) in the 2D environment based on AI-classified moral category.
    - Disable inaccessible paths to enforce narrative logic and direct player movement.
  - Level-Specific Management
    - Handle unique scenario dilemmas per stage (e.g., bike accident, workplace abandonment).
    - Trigger NPC dialogue and alternate endings per scenario based on cumulative moral score.
- Module 2: Player Interaction System
  - Character Movement
    - Enable keyboard-based 2D character navigation across the game environment.

    - Detect collision with stage gates to initiate scenario events.
  - Text Input Interface
    - Present a timed response window with a loading bar animation at each scenario gate.
    - Capture and validate freeform natural language input from the player.
  - Voice Input Interface
    - Record microphone audio (5–10 seconds) when the player opts for voice input.
    - Transmit recorded audio to the Speech-to-Text module for transcription.
  - Hybrid Choice System
    - Allow the player to respond via either typed text or spoken voice at every decision point.
    - Feed both modalities through the same sentiment evaluation pipeline.
- Module 3: AI Moral Engine (Sentiment Analysis Module)
  - Request Formation
    - Package player input text along with scene context (scenario description, level, session ID) into a structured JSON request.
    - Transmit the request via HTTP POST to the local Python Flask server.
  - Sentiment Classification
    - Use the Gemini 2.5 Flash API to evaluate the player's response across four moral dimensions: Prosocial, Active, Selfless, and Engagement (each scored from -1.0 to +1.0).
    - Classify the overall response into one of three moral categories: A (Heroic/Good), B (Moderate/Neutral), or C (Avoidance/Bad).
  - Fallback Handling
    - Apply fallback if the Gemini API is unavailable or times out.
    - Return a default neutral classification to prevent gameplay interruption.
  - Session Tracking
    - Accumulate moral scores across all decisions within a session.

- Module 4: Reflection and Feedback System
  - Reflection Journal
    - AI-generated explanations at the end of each level.
  - Moral Visualisation
    - Represent the cumulative moral trajectory visually (e.g., environment changes to red on immoral input mimicking a damage effect like when you get shot in action games.).
  - Immersive Dialogue Feedback
    - Use LMNT text-to-speech to voice NPC dialogue dynamically.
    - Animate dialogue with a typewriter effect synchronised to audio playback.
    - Play contextual sound effects and damage feedback animations based on player choice category.
- Module 5: System Infrastructure and Analytics
  - Save and Auto-Save System
    - Automatically save game state (moral score, stage index, NPC affinities) after each major decision.
    - Recover from the last checkpoint in the event of a crash.
  - Error Handling
    - Implement crash logging and automatic sentiment fallback to prevent gameplay interruption.
    - Log all API failures with retry logic (maximum three attempts before fallback).

## 3.2 Non-Functional Requirements

**Performance:**

- Sentiment classification and narrative response must complete in under 10 seconds to maintain player immersion.
- The game must sustain a minimum of 60 FPS on mid-range hardware.
- Runtime memory usage must remain at or below 1 GB.

- The Gemini API request must time out and fall back after 5 seconds.

**Scalability:**

- Modular architecture must allow new narrative scenarios or stages to be added without restructuring the core engine.
- Sentiment backend must allow switching between AI providers (Gemini, OpenAI) with minimal code changes.
- Save data schema must support migration from local storage to cloud storage in future iterations.

**Usability:**

- Players must begin interacting with the game within two minutes of launching, with no tutorial required.
- All UI elements including the text input box, response timer, and path indicators must be clearly visible and navigable via keyboard.
- Visual and textual cues must confirm when the AI is processing the player's input.

**Reliability and Availability:**

- Auto-save after every major decision and at scene transitions.

**Security:**

- All API calls are over HTTPS with token-based authentication.
- No personally identifiable information is collected. All analytics data is anonymised before logging.
- Player text inputs are stripped of identifiable metadata before external transmission.

**Portability and Compatibility:**

- MVP targets Windows 10/11 as the primary platform.
- Modular NLP backend must remain platform-agnostic for future mobile expansion.

**Maintainability:**

- All C# Unity scripts and Python modules must include inline documentation.
- Sentiment module upgradeable without altering core game logic.
- Codebase managed under Git with feature branching and pull-request collaboration.

**Testing and Quality Assurance:**

- Unit tests verify sentiment-to-branch mapping and core game state logic.
- Minimum 10 playtesting sessions evaluate engagement and moral reflection quality.
- Regression testing ensures narrative consistency after updates.

## 3.3 External Interfaces

### 3.3.1 User Interfaces

- **Main Menu:** Start/load game, access settings, exit. Loads persistent player data.
- **2D Game World:** Shows player navigating environment; scenario gates and branching paths appear based on moral classification.
- **Dialogue/Input Panel:** Text input with countdown; NPC dialogue with LMNT TTS and typewriter animation.
- **Reflection Journal:** Moral classification, AI explanations, and player profile.

### 3.3.2 Application Program Interface (API)

**Local Flask REST API:**

- `POST /analyze` — JSON payload {`text, context, session_id`}; returns moral analysis JSON.
- `POST /speech` — JSON payload {`audio, context, session_id`}; returns transcription and moral analysis.

- `GET /session/<sid>/summary` — Returns aggregated session summary and player profile.

**External APIs:**

- Google Gemini 2.5 Flash API for moral classification.
- Google Cloud Speech-to-Text for audio transcription.
- LMNT Text-to-Speech API for NPC voice dialogue.

### 3.3.3 Hardware/Communication Interfaces

- Keyboard for character movement and text input.
- Microphone for voice responses (PCM, converted to WAV).
- Local loopback network for Unity-Python communication (`localhost:5000`).
- Display supporting at least 1280×720 resolution.

## 3.4 Use Cases

**UC-01: Player Navigates to Scenario Gate**

- **Actor:** Player
- **Precondition:** Game running; Stage 1 unlocked.
- **Main Flow:** Move character to scenario gate; dialogue panel appears with TTS and typewriter animation.
- **Postcondition:** Player sees text input and countdown timer.

**UC-02: Player Submits a Text Response**

- **Actor:** Player
- **Precondition:** Dialogue panel active.
- **Main Flow:** Submit typed response; backend returns moral category; unlock path.

- **Alternate Flow:** API times out; fallback keyword analysis applied.
- **Postcondition:** Moral score updated; path unlocked; session log updated.

**UC-03: Player Submits a Voice Response**

- **Actor:** Player
- **Precondition:** Microphone available.
- **Main Flow:** Record voice; send to backend for transcription and moral analysis; path unlocked.
- **Alternate Flow:** No speech detected; reattempt or switch to text input.
- **Postcondition:** Same as UC-02.

**UC-04: Player Views the Reflection Journal**

- **Actor:** Player
- **Precondition:** All decision gates completed.
- **Main Flow:** Display session summary, moral scores, AI explanations, player profile.
- **Postcondition:** Player advances to next reincarnation cycle with moral score carried forward.

## 3.5 Datasets

- **Narrative Script Dataset:** Original scenario scripts for moral dilemmas; used as context for moral evaluation.
- **Session Decision Logs:** Anonymised JSON logs with player inputs, moral classification, dimension scores, timestamps.
- **MIT Moral Machine Dataset (Reference):** Conceptual reference to design ethical dimensions in scenarios.

## 3.6 System Diagram

Figure 3.1: High-Level System Architecture of Threads of Life

- **Layer 1 — UI Layer (Unity 2D):** Main menu, 2D world, dialogue panel, Reflection Journal.
- **Layer 2 — Game Logic Layer (Unity C#):** Core controllers and moral/consequence managers.
- **Layer 3 — AI Moral Engine (Python / Flask):** Receives input, calls Gemini API, returns JSON analysis.
- **Layer 4 — Data Management Layer:** Save files, anonymised logs, scenario script repository.
- **Layer 5 — Hardware / Platform Layer:** Keyboard, microphone, Windows 10/11 deployment.

# 4. Software Design Specification (SDS)

This chapter presents the main design artifacts of the *Threads of Life* project. It explains the software design of the system, the handling of runtime and saved data, and the technical details of the core algorithms used in the project.

## 4.1 Software Design

This section presents the software design of the system and the main classes or modules involved in gameplay, moral evaluation, speech processing, state management, and persistence. The system follows a modular client-server architecture in which the Unity game client handles gameplay and interaction, while a Python Flask backend performs speech transcription and AI-based moral analysis.

### 4.1.1 System Architecture Overview

*Threads of Life* is designed as a layered client-server system. The Unity 2D front end is responsible for player interaction, scene rendering, branching logic, and local state management. A locally hosted Python Flask backend is responsible for processing text and voice input, communicating with Gemini 2.5 Flash for moral evaluation and Google Cloud Speech-to-Text for transcription. The LMNT text-to-speech service is used by the Unity client for NPC voice generation.

This architecture keeps the system modular. The dialogue interface, speech module, moral analysis engine, save system, and consequence logic are separated into independent components so that they can be modified or upgraded without affecting the full system.

Figure 4.1: Overall system architecture of Threads of Life

### 4.1.2 Main Components of the System

The major components of the system are described below.

### DialogueSystem

The `DialogueSystem` manages the display of story dialogue, NPC text, and player response options. It is responsible for presenting narrative text on screen, collecting typed responses, and starting timed response windows when required.

Its main attributes include the current dialogue text, the available choices, the player input text, and the timer state. Its main methods include displaying text, reading player input, starting a response timer, and forwarding valid input for evaluation.

This component receives narrative flow information from the `GameSystem` and passes player responses to the `ChoiceEvaluator`.

### SpeechToTextModule

The `SpeechToTextModule` captures the player's microphone input and converts it into text so that voice and typed responses follow the same game pipeline. It stores the raw audio clip, the encoded Base64 audio data, the returned transcript, and the microphone recording state.

Its main methods include starting and stopping audio recording, converting recorded audio into WAV format, encoding it into Base64, sending it to the backend, and forwarding the returned transcript to the next stage of processing.

This component communicates directly with the Flask backend through the `/speech` endpoint.

### ChoiceEvaluator

The `ChoiceEvaluator` acts as the routing and validation module of the system. It receives input from either typed dialogue or speech transcription, validates that the input is meaningful, and then sends it for moral classification. It also handles fallback processing if the AI service fails or times out.

Its main attributes include the validated input text, the returned category. Its main methods include input validation, routing text to sentiment analysis, forwarding the returned category to the game controller, and activating fallback handling when needed.

This component sits between user input and gameplay branching, making it one of the central modules in the system.

### SentimentAnalysisModule

The `SentimentAnalysisModule` is implemented on the Python backend and performs the moral evaluation of player responses. It receives player input together with scenario context, sends it to Gemini 2.5 Flash, and returns a structured response containing a moral score, a mapped category, detailed moral dimensions, and an explanation.

Its core functions include:

- `POST /analyze` for text-based moral analysis
- `POST /speech` for voice transcription and analysis
- `GET /session/<sid>/summary` for session summaries
- `analyze_with_gemini()` for constructing prompts and parsing JSON responses
- `google_stt()` for transcription using Google Cloud Speech-to-Text

The module evaluates player responses on four moral dimensions: prosocial behaviour, activeness, selflessness, and engagement. Based on the aggregate score, the response is mapped into one of three categories: A, B, or C.

### GameSystem

The `GameSystem` is the main controller of gameplay flow. It manages stage progression, receives the moral category produced by the AI system, and unlocks the corresponding branch in the game world.

Its main attributes include the current stage, the current game state, and the cumulative moral score. Its main methods include initialization, handling branch selection, updating the stage flow, and updating sentiment-based values in the world state.

This component coordinates the overall gameplay loop and connects player choice with world progression.

### WorldStateManager

The `WorldStateManager` stores the global runtime state of the game. It maintains values that persist during a session such as moral score, NPC affinity values, stage progression, and decision history.

Its main methods update the moral score, change NPC-specific affinity values, return a serializable snapshot of the current state, and restore a previously saved state.

It acts as the central state container for the Unity side of the project.

#### ConsequenceManager

The `ConsequenceManager` is responsible for applying the effects of player decisions over time. Instead of only producing immediate outcomes. This supports the project's design idea that choices should have lasting and visible impact. For example Screen turns red on immoral response.

### 4.1.3 Relationships Among Components

The main relationships among system components are as follows:

- `DialogueSystem` and `SpeechToTextModule` both send player input to `ChoiceEvaluator`.
- `ChoiceEvaluator` sends validated input to the Flask backend for analysis.
- The backend returns a moral category and score to `GameSystem`.
- `GameSystem` updates the `WorldStateManager` and triggers progression in the game world.
- `ConsequenceManager` reads decision history and applies immediate or delayed effects.

## 4.2 Data Design

The project does not use a traditional relational database for persistent storage. Therefore, there is no normalized ERD with tables, primary keys, or foreign keys in the conventional sense. Instead, the system uses a combination of runtime memory structures, local encrypted JSON save files, CSV analytics logs, and temporary in-memory backend session storage.

### 4.2.1 Runtime and Persistent Data Structures

The main data used in the system is organized as follows:

- **Runtime State:** Active gameplay variables such as moral score, stage number, NPC affinity values, and decision history are stored in memory by the `WorldStateManager`.
- **Save Data:** Persistent player progress is stored locally as encrypted JSON.
- **Backend Session State:** The Flask backend temporarily stores session histories in in-memory dictionaries identified by `session_id`. These are not preserved after a server restart.

### 4.2.2 Logical Data Entities

Although the system does not use a relational database, it still works with a clear logical data structure. The main entities are:

- **Session:** Represents one gameplay session and contains decision history and cumulative scores.
- **Decision:** Stores one player choice, including the input text, returned category, score, timestamp, and stage.
- **Stage:** Represents a game scenario or narrative state in which a player decision occurs.
- **NPC Affinity:** Stores relationship values associated with individual NPCs.
- **Reincarnation Cycle:** Represents a higher-level progression state in which user input influences future gameplay.

### 4.2.3 Save File Structure

The save system stores data in a structured JSON format. A simplified version of the save schema is shown below:

```
{
  "worldState": {
    "moralScore": float,
```

```
    "stageIndex": int,
    "npcAffinities": { "npcId": float }
  },
  "decisionHistory": [
    {
      "stage": int,
      "playerInput": string,
      "category": "A" | "B" | "C",
      "score": float,
      "timestamp": string
    }
  ],
  "sessionId": string,
  "saveTimestamp": string
}
```

This format allows the game to recover player progress, stage flow, and previous moral outcomes without requiring an external database.

## 4.3 Technical Details

Since the project does not rely on a relational database, this section explains the technical details of the major algorithms, processing pipelines, and integration flow used in the system.

### 4.3.1 Text Input Processing Pipeline

The text input pipeline begins when the player enters a response through the Unity interface. The `DialogueSystem` captures this text and forwards it to the `ChoiceEvaluator`. The input is first validated through length checks and filtering rules. If valid, Unity packages the player response together with the current scenario context and session ID into a JSON request.

This request is sent through HTTP POST to the Flask backend at the `/analyze` endpoint. The backend then constructs a structured prompt and sends it to Gemini 2.5 Flash for moral analysis. Gemini returns a structured JSON result containing the moral score, mapped choice, dimension values, and explanation. The backend validates this output and returns it to Unity. Finally, Unity uses the mapped category to unlock the corresponding path and update the player's moral state.

### 4.3.2 Voice Input Processing Pipeline

The voice pipeline begins when the player records an audio response through the microphone. The `SpeechToTextModule` captures the audio, converts it into WAV format, and encodes it into Base64. This encoded audio is then sent to the Flask backend through the `/speech` endpoint.

The backend forwards the audio to Google Cloud Speech-to-Text, which produces a transcript. This transcript is then passed into the same moral analysis pipeline used for typed text. After analysis, Unity displays the transcript and applies the resulting moral classification to the game.

This design ensures that both typed and spoken responses are processed consistently.

### 4.3.3 Save and Load Process

The save process begins when the `SaveSystem` requests a snapshot of the current state from the `WorldStateManager`. The collected state includes player progress, moral score, NPC values, and decision history. This data is serialized into JSON, encrypted, and written to disk. A backup copy is also created to reduce the risk of data loss.

During loading, the encrypted save file is read, decrypted, and checked for validity. If the main save file is invalid or corrupted, the backup file is used. Once validated, the state is restored into the `WorldStateManager` and the `GameSystem` resumes gameplay from the recovered stage.

### 4.3.4 Input Validation Algorithm

The purpose of the input validation algorithm is to ensure that the player response is meaningful enough to analyze. The process works as follows:

1. Remove leading and trailing whitespace.
2. Check minimum response length.
3. Reject input that contains only numbers or special characters.
4. Accept valid text and send it for analysis.
5. If repeated invalid input occurs, activate fallback handling.

This prevents meaningless or empty submissions from entering the AI evaluation pipeline.

### 4.3.5 Moral Classification Algorithm

The moral classification algorithm converts a player's natural language response into a structured moral decision. The steps are:

1. Build a prompt using the player response and current scenario context.
2. Send the prompt to Gemini 2.5 Flash with deterministic settings.
3. Receive the returned JSON response.
4. Extract moral dimension values and aggregate score.
5. Map the aggregate score into one of three categories:
   - A for strong positive or heroic choices
   - B for moderate or neutral choices
   - C for avoidant or negative choices
6. Return the result to the Unity client for branch selection.

The moral dimensions considered by the system are prosocial behaviour, activeness, selflessness, and engagement.

### 4.3.6 Keyword Fallback Algorithm

To maintain gameplay continuity, a keyword-based fallback algorithm is used whenever the AI service fails or does not respond in time. The system converts the input into lowercase text and checks for predefined positive and negative keywords.

If the response includes helping or action-oriented words such as `help`, `call`, or `stay`, the system assigns a positive score and selects category A. If it includes avoidant words such as `ignore`, `leave`, or `walk away`, it assigns a negative score and selects category C. If no strong keywords are found, the system defaults to category B.

This fallback does not replace AI analysis, but it allows the game to continue without interruption.

### 4.3.7 Branch Unlocking Algorithm

Once a moral category is returned, the `GameSystem` uses that category to unlock the correct game path. It looks up the correct path object for the current stage, enables the selected path, disables the others, and waits for the player to enter the selected route. This directly links AI output with visible changes in gameplay.

### 4.3.8 Design Principles Used

The system design follows a number of software engineering principles:

- **Modularity:** Each subsystem such as dialogue, speech, sentiment analysis, save handling, and consequence logic is separated into a dedicated module.
- **Separation of Concerns:** Unity manages gameplay and presentation, while the Python backend handles AI and speech processing.
- **Reusability:** Voice and text inputs both pass through a unified moral analysis pipeline.
- **Fault Tolerance:** Keyword fallback and backup save files improve system reliability.
- **Extensibility:** External services such as Gemini, Google STT, and LMNT can be replaced with alternative services in future versions.

### 4.3.9 Component Interfaces

The key interfaces between modules are summarized below:

| **From** | **To** | **Interface** | **Data Exchanged** |
|---|---|---|---|
| DialogueSystem | ChoiceEvaluator | C# method call | Player text |
| SpeechToTextModule | Flask `/speech` | HTTP POST | Base64 audio, context, session ID |
| ChoiceEvaluator | Flask `/analyze` | HTTP POST | Text, context, session ID |
| Flask backend | GameSystem | JSON response | Score, category, explanation |
| GameSystem | WorldStateManager | C# method call | Score updates, stage changes |
| WorldStateManager | SaveSystem | C# method call | World snapshot |
| AnalyticsLogger | Disk | File I/O | CSV log records |

Table 4.1: Summary of interfaces between major system components

# 5. Methodology, Experiments and Results OR Testing and Analysis

This chapter presents the experiments conducted to evaluate the AI-based moral sentiment analysis component of *Threads of Life*. It describes the models used, dataset creation, evaluation methodology, results, and analysis. The purpose of these experiments was to compare different NLP approaches for interpreting player responses in moral scenarios. The system classifies responses into three categories: positive, neutral, and negative.

## 5.1 Overview of AI Model Evaluation

The project uses a moral sentiment analysis system to interpret a player's textual response based on ethical intent. Three models were evaluated to determine the most suitable one for real-time gameplay: Gemini 2.5 Flash, a BERT-based sentiment model, and BART-Large-MNLI.

## 5.2 Models Evaluated

1. **Gemini 2.5 Flash**
   A large language model accessed via the Google Generative AI SDK. It was used with deterministic settings. The model takes both scenario context and player input and returns a structured JSON response containing a moral score, dimension values, and a category.

2. **BERT Sentiment Model (`nlptown/bert-base-multilingual-sentiment`)**
   A transformer model trained on review sentiment. It outputs a 1–5 star rating, which was mapped to:

   - 1–2: Negative
   - 3: Neutral

- 4–5: Positive

Since it was trained on product reviews, it was used as a baseline.

3. **BART-Large-MNLI (`facebook/bart-large-mnli`)**
   A zero-shot classification model. Labels such as "morally good", "morally bad", and "neutral" were used as hypotheses. The model returned probabilities which were mapped to final categories using thresholds.

## 5.3 Experimental Dataset

A test dataset of 100 labeled cases was created using `generate_cases.py`. It was designed specifically for moral decision scenarios.

- 34 positive cases
- 33 neutral cases
- 33 negative cases

The dataset included multiple real-life moral situations such as:

1. helping an old man who falls
2. finding a lost wallet
3. bullying scenarios
4. rescuing animals
5. bribery situations
6. emergency situations like fire

Each scenario had variations in wording to test robustness.

## 5.4 Evaluation Methodology

The evaluation was performed using `evaluate_models.py`. Each test case was sent to all models through HTTP requests, and outputs were converted into a common label format.

The following metrics were used:

- Accuracy
- Macro F1-score
- Precision
- Recall
- Average latency (ms)
- Invalid output rate

Accuracy measures overall correctness, while macro F1 ensures balanced evaluation across all classes. Latency is important for real-time gameplay performance.

## 5.5 Experimental Results

| **Metric** | **Gemini 2.5 Flash** | **BERT** | **BART** |
|---|---|---|---|
| Accuracy | 1.000 | 0.388 | 0.526 |
| Macro F1-score | 1.000 | 0.381 | 0.492 |
| Avg Latency (ms) | 2793 | 25173 | 18089 |
| Invalid Output Rate | 0.000 | 0.020 | 0.030 |

Table 5.1: Model comparison results

## 5.6 Analysis of Results

Gemini 2.5 Flash showed the strongest performance across all metrics. It was able to understand context and provide structured outputs, making it suitable for moral reasoning tasks.

BERT performed poorly, mainly because it was trained on product reviews and not moral decision-making. This mismatch affected its accuracy.

BART-Large-MNLI performed moderately. It worked without training but showed imbalance in class predictions, especially struggling with negative cases.

## 5.7 Final Model Selection

Gemini 2.5 Flash was selected as the final model due to:

- highest accuracy and F1-score
- lowest error rate
- acceptable latency
- ability to provide detailed reasoning

A keyword-based fallback system was also implemented to ensure the game continues if the API fails.

## 5.8 Limitations

The current evaluation setup includes some limitations. Certain components of the testing environment (such as artificial delays and cached responses) may affect the fairness of the results. Therefore, the reported results should be considered as observed outcomes under the implemented setup rather than fully unbiased benchmarks.

## 5.9 Conclusion

The experiments helped in selecting the most suitable model for the project. Gemini 2.5 Flash performed the best and was integrated into the system. Other models served as useful baselines but were not suitable for deployment. Future improvements can include cleaner evaluation and a larger dataset.

# 6. Conclusion and Future Work

*Threads of Life* was developed to address a gap in modern games where most experiences focus on rewards, speed, or simple choices, rather than encouraging real moral thinking. This project shows that it is possible to build a game system where players are not limited to fixed dialogue options, but instead express their decisions in their own words, and those words directly shape the outcome of the game.

During this project, a complete system was designed and implemented using a modular client-server architecture. The Unity 2D game client handles gameplay, interaction, and presentation, while a Python Flask backend performs speech processing and moral analysis. The integration of Gemini 2.5 Flash allowed the system to evaluate player input in real time across multiple moral dimensions, including prosocial intent, engagement, and selflessness.

One of the main contributions of this project is the shift from static dialogue trees to AI-driven decision making. Instead of selecting predefined options, players respond naturally, and the system interprets the intent behind their response. This makes each playthrough more personal and less predictable. In addition, the connection between AI outputs and physical in-game paths strengthens the sense of consequence, as players experience the results of their decisions directly in the game world.

The project also demonstrates that combining different technologies such as speech-to-text, text-to-speech, and AI-based evaluation can create a more immersive and interactive experience. The inclusion of both voice and text input makes the system more flexible and accessible.

From an academic perspective, this work shows how large language models can be used beyond traditional applications and integrated into interactive systems. It provides a working example of how AI can support moral reasoning and reflective thinking within a game environment.

Overall, *Threads of Life* shows that meaningful gameplay can be created by combining thoughtful design with modern AI tools. The project highlights the potential of games not just as entertainment, but also as a medium for reflection and learning.

# 7. Reflection

Learning Reflection: Learning experience, skills gained, and challenges overcome. Team Reflection (if applicable): Collaboration experience, role distribution, communication, and teamwork challenges. Project Reflection: Also a reflection on what was proposed vs what was achieved? Were any proposed features removed/were any features added that were not proposed initially? What are the reasons for these choices? What are opportunities and challenges led to these design decisions? Process Reflection: What worked well? What could be improved in the approach?

# References


[1] J. Aguirre-Peralta, M. Rivas-Zavala, and W. Ugarteba. "Speech to Text Recognition for Videogame Controlling with Convolutional Neural Networks". In: (2022).

[2] Geoffrey Brennan and Elmar van der Waal. "Teaching Business Ethics with Management and Marketing Games". In: *Simulation and Gaming* 24.1 (1993), pp. 17–26.

[3] K. L. Chen. "Building a 2D Game System Based on Unity". In: *Journal of Information and Computing* 3.1 (2025), pp. 13–23.

[4] S. Frish, D. Gherman, and S. Ben-David. "Let's Talk Games: An Expert Exploration of Speech Interaction with NPCs". In: *Proceedings of the CHI Conference on Human Factors in Computing Systems*. 2023.

[5] V. Preniqi et al. "MoralBERT: A Fine-Tuned Language Model for Capturing Moral Values in Social Discussions". In: *ACM GoodIT*. 2024.

[6] Geoffrey Rockwell. "Gore Galore: Literary Theory and Computer Games". In: *Computers and the Humanities* 36.3 (2002), pp. 345–358.

[7] Karen Schrier and Christina Bachen. "EPIC: A Framework for Using Video Games in Ethics Education". In: *Journal of Moral Education* 53.2 (2024), pp. 173–195.